

Blockchain-Based Edge Computing Service With Dynamic Entry and Exit Mechanism

Qiang He , Zheng Feng , Hui Fang , Xingwei Wang , *Member, IEEE*, Liang Zhao ,
Keping Yu , *Member, IEEE*, and Kim-Kwang Raymond Choo , *Senior Member, IEEE*

I. INTRODUCTION

Abstract—With the widespread application of 5G and artificial intelligence (AI) technology, the Internet of Things (IoT) has been expanding and integrated into various aspects of our daily lives. However, this also poses challenges such as the ubiquitous demand for communication and computing resources, and data privacy issues. Considering its flexible deployment, high security, and ease of scalability, blockchain-enabled edge computing IoT network (BECIN) has become a promising solution to provide secure and fast communication and computing services. However, existing research on computation offloading in edge computing largely overlooks the stochastic arrival of computational tasks and the potential variability in the number, locations, and resource provisions of edge computing service providers. Therefore, we propose a dynamic, self-adjusting BECIN framework aimed at providing long-term stable, efficient, and secure edge computing data offloading services for ground users in a specific region. This framework supports the dynamic entry and exit of edge computing service providers. Additionally, we introduce a novel dynamic Dueling DDQN approach to update the offloading and resource management policies based on changes in resource provisioning. Experimental results demonstrate the feasibility and superior performance of our framework on system cost and system latency.

Index Terms—Internet of Things networks, Edge computing, entry and exit mechanism, deep reinforcement learning.

Received 17 January 2025; revised 11 April 2025; accepted 9 May 2025. Date of publication 16 May 2025; date of current version 3 September 2025. This work was supported in part by the Major International (Regional) Joint Research Project, in part by the National Natural Science Foundation of China under Grant 71620107003, in part by the National Natural Science Foundation of China under Grant 72371148, Grant 72192832, Grant 62432003, and Grant U22A2004, in part by the Shanghai Rising-Star Program under Grant 23QA1403100, in part by the Fundamental Research Funds for the Central Universities under Grant N25QNR006, and in part by the National Key R&D Program of China under Grant 2022YFB4500800. Recommended for acceptance by H. T. Dinh. (Corresponding author: Liang Zhao.)

Qiang He is with the College of Computing and Artificial Intelligence, Northeastern University, Shenyang 110819, China (e-mail: heqiang@bmie.neu.edu.cn).

Zheng Feng is with the College of Medicine and Biological Information Engineering, Northeastern University, Shenyang 110819, China.

Hui Fang is with the Research Institute for Interdisciplinary Science and School of Information Management and Engineering, Shanghai University of Finance and Economics, Shanghai 200437, China.

Xingwei Wang is with the College of Computer Science and Engineering, Northeastern University, Shenyang 110169, China.

Liang Zhao is with the College of Computer Science and Engineering, Shenyang Aerospace University, Shenyang 110136, China (e-mail: lzhaol@sau.edu.cn).

Keping Yu is with the Graduate School of Science and Engineering, Hosei University, Tokyo 184-8584, Japan.

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78249 USA (e-mail: raymond.choo@fulbright-mail.org).

Digital Object Identifier 10.1109/TMC.2025.3570646

WITH the rapid proliferation and increasing intelligence of Internet of Things devices, the amount of generated data has grown sharply. Related report has shown that the global IoT device count will reach 75 billion by 2025 [1]. However, such a massive number of devices produces a huge amount of data, and processing that data only using cloud servers would result in significant delays and energy consumption. Recently, the edge computing IoT network (ECIN) [2], [3], [4] is an emerging computing architecture that implements distributed computing for data processing and task execution by deploying edge nodes at the end of IoT devices. In contrast to traditional cloud computing, ECIN offers lower latency, higher real-time capability, reduced network load, decreased bandwidth requirements, and enhanced data privacy and security. With the advancement of edge computing and the Internet of Things, ECIN plays a critical role across various domains, driving innovation and development in IoT applications. Edge computing enables Industry 5.0 to access and exchange information across industrial sectors through more accessible standard hardware and software resources [5]. Additionally, it empowers smart manufacturing by providing portable bearing fault diagnostics and remote localization methods [6], while also promoting proactive edge computing in smart cities, thus exploring new IoT use cases driven by machines [7].

However, using edge computing to transfer data processing to the local end still poses security risks, such as physical security and data privacy of edge computing nodes. Transmitting local data to the edge for processing may result in security issues such as data leakage, identity fraud, and data tampering. Data interception or hijacking during transmission can lead to data leakage or tampering, which may adversely affect users' privacy and sensitive information. Additionally, edge computing nodes may be susceptible to various attacks, such as malware, network attacks, and denial-of-service attacks, which may cause data loss or node failure, thus affecting system reliability and security [8], [9]. Therefore, it is necessary to develop a secure edge computing architecture that ensures data security, prevents data tampering or loss, and protects edge computing nodes from various attacks.

Blockchain is a promising technology that leverages a peer-to-peer network topology with immutable, transparent, and tamper-resistant data ledger [10], [11], [12]. By integrating blockchain into the ECIN architecture, the BECIN framework is created to achieve reliable decentralized edge computing, which ensures data immutability and decentralization while enabling

identity verification and access control. Furthermore, blockchain facilitates data sharing and collaboration to safeguard data security and privacy. Specifically, the BECIN framework leverages smart contracts to automate access control and enforce service-level agreements among edge nodes, enabling secure and autonomous interactions without manual intervention. To achieve decentralization and eliminate reliance on a centralized authority, consensus algorithms such as Proof of Stake and Practical Byzantine Fault Tolerance are employed to validate transactions and maintain a consistent global state across distributed ledgers. Moreover, cryptographic primitives including digital signatures and hash functions are incorporated to enable identity authentication, ensure data integrity, and support secure communication within the edge computing network. There is considerable research on BECIN [13], [14], [15], [16], but current studies primarily focus on several key areas, including addressing challenges such as communication security, privacy protection, critical vulnerabilities, high processing loads on cloud servers, and increasing latency [17]. For example, these technologies can be applied in intelligent transportation systems to enable secure collaboration among multiple edge nodes while maintaining privacy [18]. Additionally, in IoT environments, blockchain-supported edge computing enhances secure message transmission between devices [19]. In addition to these, many researchers have considered addressing challenges such as high energy consumption and latency, given the scale and complexity of BECIN [20]. These studies usually neglect the potential variability in computing task arrivals and the number, location, and resource availability of edge computing service providers. In practical applications, the arrival time and quantity of computing tasks are stochastic due to the extensive number and dispersed distribution of IoT devices. Additionally, the variability of the number of edge computing service providers is also exceedingly high. For instance, scenarios involving the addition of new edge servers and the failure of existing ones may arise. Moreover, the location and resource availability of edge computing service providers may change. For instance, some edge nodes may require relocation or adjustment due to high communication delays caused by geographical location or other factors. With the expansion of edge computing services, the computing and communication resources provided by edge computing nodes may also be changed, such as increasing or decreasing the number of computing nodes, adjusting node computing resource allocation, or modifying node communication bandwidth allocation.

Furthermore, the variability in the number of edge computing service providers requires security measures to control the joining and exiting of nodes. Firstly, malicious users or unauthorized nodes may join the edge computing network and gain access to sensitive data or carry out malicious operations without verification identification and permission control mechanisms. Secondly, a single participant can manipulate the decisions regarding node joining and exiting in the absence of multi-party signature authentication, such as forcing legitimate nodes to exit or allowing unverified nodes to join, thereby undermining the fairness and reliability of the network. Additionally, blockchain technology provides immutable transaction records that can be utilized for the supervision and auditing of the node joining

and exiting processes. This transparency enables participants to monitor the operation of the network, identify abnormal behavior, and take appropriate measures. Therefore, a dynamically adjustable BECIN framework with regulatory characteristics is of significant importance for real-world business applications. It is expected to facilitate the dynamic entry and exit of edge computing service providers and updates their service policies in response to the changes of resource supply. And, the primary objective of BECIN is to provide long-term, stable, and secure edge computing services to ground users in specific regions.

A. Related Work

We have summarized the related work on latency, energy optimization, and resource allocation for both standard ECIN and decentralized BECIN.

Standard ECIN: The decision-making process regarding task offloading is a prominent topic in ECIN research, especially with regards to reducing latency. In this context, a software-defined task offloading scheme was proposed in [21] to minimize latency while conserving the battery life of user devices. Zhang et al. [22] investigated collaboration-based task offloading and data caching models to minimize overall latency across all mobile devices. Furthermore, Bai et al. [23] considered the joint optimization of computation offloading, edge computing resources, multiple user detection matrices, and intelligent reflecting surfaces to minimize the weighted computation delay of all devices.

On the other hand, offloading decisions by reducing energy consumption have also attracted attention. For example, You et al. [24] formulated the resource allocation problem in a mobile edge computing offloading system to minimize both weighted energy consumption and mobility under computation delay constraints with infinite or finite edge cloud computing capacity. Ding et al. [25] investigated the mobility problem of user devices, including random mobility, short-term predictable mobility, and fully known mobility, to optimize task offloading and service migration delay. Xu et al. [26] considered the inference offloading problem driven by deep neural networks in 5G-enabled mobile edge computing, aiming to minimize energy consumption of mobile devices, cloud computing, and 5G base stations or maximize the number of accepted inference requests.

For resource allocation, Fan et al. [27] explored the joint WiFi and cellular offloading problem for mobile terminals to balance energy consumption and latency objectives. The authors considered task offloading on different priority channels and defined the objective function to achieve the trade-off between energy consumption and delay for task processing. Luo et al. [28] defined the quality of experience (QoE) for users as a weighted sum of latency and energy consumption, and solved the offloading scheduling problem by jointly allocating communication and computing resources while considering user QoE. Moreover, Li et al. [29] designed a novel physical-layer-based utility function that measured the cost reduction achieved by offloading compared to local execution of tasks. Two computation offloading decision games were formulated using this utility function by maximizing individual benefits and overall system benefits.

Decentralized and Blockchain-Based ECIN: Recent research has explored the use of decentralized ECIN schemes for security and fairness. For instance, in [30], the authors proposed a decentralized computation offloading algorithm, aimed at minimizing the average task completion time in edge computing networks and addressing fairness issues among different edge devices in a fully decentralized environment. Another study in [31] developed an economically efficient home health monitoring system, utilizing decentralized non-cooperative game theory to minimize system-wide costs. Edge computing has also provided a decentralized and secure alternative for large-scale machine learning, as demonstrated in [32], allowing for learning model parameters in a multi-agent system where data was processed locally by distributed edge nodes. Additionally, some studies have employed decentralized methods to solve the data offloading problem in edge computing, such as the decentralized responsive method proposed in [33], which employed fuzzy control mechanism to generate real-time decisions based on long-term allocation rates, leasing costs, and energy consumption. In [34], the use of federated learning in edge computing was introduced. To address the convergence speed issue of existing algorithms, the authors proposed a novel algorithm called CoCo, which utilized adaptive topology structures and model compression techniques to accelerate the convergence speed of distributed federated learning.

Recently, integrating blockchain into ECIN has been studied. DeTEC, a new decentralized and trusted edge computing platform that combines blockchain technology with edge computing has been proposed in [35]. DeTEC solved the task allocation problem in edge computing by allocating user requests to the most suitable edge servers through a domain name server, and incentivized node participation through a reward mechanism. Liu et al. [36] introduced a blockchain-based mobile edge computing architecture to build a decentralized video ecosystem for distributed and secure delivery of video streams. The authors proposed a problem-solving algorithm based on a three-stage Stackelberg game. Yuan et al. [37] proposed CoopEdge+, a blockchain-based distributed platform that promoted and supported collaborative computing in edge computing to address challenges such as incentive and trust in collaborative computing among edge servers in a dispersed environment. A collaborative bidding and recording model was implemented on CoopEdge+, and its practicality and performance were validated through experiments. In [38], the authors used blockchain to implement a decentralized resource allocation and pricing mechanism, and designed a differential privacy combined double auction mechanism to ensure privacy and maximize the revenue of the edge computing platform.

B. Motivations and Key Contributions

Although significant research efforts have been made, there are still limitations to current BECIN works, which are highlighted below:

- Current works [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34] seldom consider the inevitable unstable propagation conditions in reality,

including the randomness of computing task arrival and unstable channels when solving practical data offloading problems, which is more practical and worthy of attention.

- Existing ECIN and BECIN studies [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38] usually ignore the fact that resource supply of edge computing service providers may also change. Due to the dynamic voltage and frequency scaling technology, mobile intelligent devices and edge servers can dynamically adjust the CPU cycle frequency to reduce actual power consumption. Thus, the resource supply of edge nodes may be changed over time. Currently, research based on the BECIN architecture [35], [36], [37], [38] has rarely addressed the mechanisms for the inclusion and removal of blockchain nodes. We contend that considering the addition or withdrawal of edge computing nodes in BECIN is indispensable. In real-world scenarios, situations such as the addition of new edge nodes or the failure and subsequent exit of existing edge nodes can occur.

To address these issues, we propose a dynamic BECIN framework with self-adjusting capabilities, whose target is to provide long-term stable and secure edge computing services for ground users in specific areas. It supports the dynamic entry and exit of edge computing service providers and updates their service strategies based on changes in resource supply. Additionally, we provide an efficient and load-balanced task allocation scheme. Therefore, the main contributions of this paper are summarized as follows:

- 1) We consider a blockchain-enabled edge computing IoT network architecture that supports dynamic changes for tasks and resources, which thus considers the randomness of task arrivals and the potential variability in the number, location, and resource supply of edge computing service providers.
- 2) In this study, we propose a verifiable threshold secret sharing scheme based on the Chinese Remainder Theorem for secure data sharing and computation among blockchain-enabled edge computing servers, supporting dynamic participant join/leave operations. Furthermore, we present Dueling DDQN, a reinforcement learning approach that optimizes learning in dynamic environments. This method enhances learning stability by mitigating Q-value overestimation and improves policy adaptation through precise state-action value estimation. Our research makes a novel contribution by integrating dynamic join/leave mechanisms into the reinforcement learning framework, offering an innovative solution for dynamic tasks.
- 3) We provide theoretical correctness proofs, security analysis, and comparative experiments. Extensive analysis and experimental results demonstrate the feasibility and superior performance of our framework.

C. Paper Organization

The rest of this paper is organized as follows. In Section II, we provide an overview of the designed system and subsequently

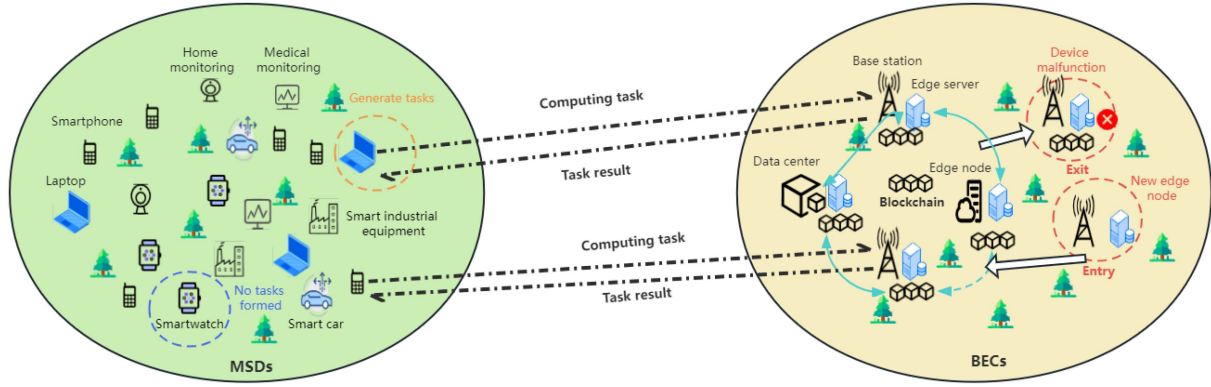


Fig. 1. Data offloading scenario in BECIN architecture (MSDs: mobile smart devices; BECs: blockchain-based edge computing servers).

define the relevant problems. In Section III, we analyze the dynamic task offloading and resource allocation problem. In Section IV, we provide a detailed description of the proposed solution. Section V shows the experimental results. Finally, we conclude the paper in Section VI.

II. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first overview the designed system, and then describe the task model, local execution model, communication and computation models.

A. System Overview

As depicted in Fig. 1, we consider a scenario of edge computing IoT empowered by blockchain, where edge computing nodes (i.e., base stations) participate as blockchain nodes in the blockchain network, providing users with secure, reliable, and environmentally friendly services. In this figure, we illustrate the potential actions that multiple edge computing nodes may undertake during a given time period under different circumstances. The scenario involves N mobile smart devices (MSDs) and M Blockchain-based edge computing servers (BECs). We denote the sets of BECs and MSDs as $\mathcal{N} = \{1, 2, \dots, N\}$ and $\mathcal{M} = \{1, 2, \dots, M\}$ respectively.

MSDs: With the rapid development of the Internet of Things, a diverse range of smart devices, including smartwatches, smart homes, smart cars, and smart industrial equipment, has become an indispensable part of people's daily lives. They are interconnected through the internet, forming a vast network that offers users numerous conveniences and intelligent functionalities. Within this network, these smart devices generate a considerable number of computational tasks. These tasks may involve complex data processing, such as the collection, analysis, and interpretation of sensor data. For instance, smart cars can acquire real-time road and traffic information through perception sensors and perform intricate data processing to optimize the driving experience [39]. Additionally, some computational tasks necessitate real-time processing and response. For example, smart health monitoring devices can continuously monitor users' physiological parameters and send emergency notifications upon detecting anomalous conditions [40].

BECs: Blockchain-based edge servers are typically located closer to user devices, in proximity to the source of data generation. They can be placed in physical base stations, data centers, edge nodes, and other locations to better meet user demands and reduce network latency. Edge servers can perform real-time data processing, analysis, and inference, reducing the need to send data to the cloud for processing. This is highly beneficial for real-time decision-making and data-driven applications in large-scale sensor networks, industrial automation, intelligent transportation, and similar scenarios. As a key component of edge computing, edge servers establish a distributed computing ecosystem. They collaborate with other edge devices, sensors, and cloud servers, supporting diverse application scenarios and innovations. When users require computational tasks, multiple mobile edge computing servers efficiently provide computing services. These servers are located in proximity to the users and establish connections with their devices using near-field communication technologies. Through intelligent resource scheduling and allocation algorithms, servers can allocate user requests to the most suitable server for processing, achieving load balancing and optimal resource utilization. Moreover, collaborative computation among multiple servers can be conducted, enabling distributed computing and data transfer, resulting in enhanced task processing efficiency and computational capacity.

As shown in Fig. 1, the arrival of computing tasks for MSDs can be influenced by various factors, such as user behavior, network conditions, and data generation rate. The variability and uncertainty of these factors contribute to the randomness of task arrivals. The randomness of computational tasks is of significant importance for system design and resource management. In a distributed computing environment, the stochastic nature of task arrivals impacts resource allocation, load balancing, and performance optimization, among others. The system needs to adapt to the uncertainty of task arrivals and employ appropriate scheduling and resource allocation strategies to handle these tasks effectively. Furthermore, the design of task scheduling and resource planning algorithms should take into account the randomness of task arrivals to ensure system efficiency and reliability.

In our simulation, we model the system as a discrete time-slot-based environment. For easier understanding, we provide

TABLE I
LIST OF KEY NOTATIONS

Variable	Description
$\mathcal{N}$	Collection of MSDs
$\mathcal{M}$	Collection of BECs
$\mathcal{K}$	Collection of subchannels
$\mathcal{T}$	Collection of time series
$T_n^L(t)$	Local computing latency
p_n^L	Local execution power
$E_n^L(n)$	Local computing energy consumption
$P_{c,m}^t$	Server computing resource allocation scheme
$P_{b,m}^t$	Server communication resource allocation scheme
$T_{c,m}^{E,t}(P_{c,m}^t)$	Computing latency at the edge
$E_{c,m}^{E,t}(P_{c,m}^t)$	Computing energy consumption at the edge
$T_{trans}^{m,n,t}(P_{b,m}^t)$	Transmission latency
$E_{trans}^{m,n,t}(P_{b,m}^t)$	Transmission energy consumption
λ_i	Average arrival rate of computational tasks
$A_n(t)$	Slot t , the number of computing tasks for MSD_n
$C_n^m(t)$	Proportion of tasks offloaded from MSD_n to BEC_m
$r_n(t), \Psi_m(t)$	MSD n /MEC m computational efficiency
$f_n^L(t)$	Local computing resources
$f_n^{m,t}(P_{c,m}^t)$	Edge computing resources
$W_n^{m,t}(P_{b,m}^t)$	Edge computing resources
k_n^L	Constant power coefficient
K_m^E	Energy-to-Power Ratio
$\mathcal{N}_m$	Set of MSDs covered by BEC_m
f_m	Computational resources of BEC_m
B_m	Bandwidth resources of BEC_m
$X_{n,k}^{m,t}$	Allocation of subchannels
$\Upsilon_{n,k}^{m,t}(P_{b,m}^t)$	Inter-cell interference
P_n^t	Transmit power of MSD_n
$H_{n,k}^{m,t}$	Channel power gain
ε	Path loss coefficient
$l_n^{m,t}$	Distance between MSD and BEC
ϑ	Path loss exponent
$h_{n,k}^{m,t}$	Power gain of small-scale fading
l_m	Location of BEC_m
l_n^t	Location of MSD_n
$\Psi_{n,k}^{m,t}(P_{b,m}^t)$	Signal-to-Interference-plus-Noise Ratio
σ^2	Noise power spectral density
$r_n^{m,t}(P_{b,m}^t)$	Transmission rate between BEC_m and MSD_n
ϕ	Weight coefficient
$T(m, n, t)$	Total delay
$E(m, n, t)$	Total energy consumption
$Q(m, n, t)$	Weighted sum of delay and energy consumption
C_{total}	Objective function

the main variable list in Table I. Time is divided into time slots, denoted as $t \in \mathcal{T} = \{0, 1, 2, \dots\}$, with each time slot having a length of T . For each mobile smart device MSD_n , the computational tasks generated in each time slot t are represented as $A_n(t)$, and the task arrivals in different time slots are assumed to be independent and identically distributed. To capture the stochastic nature of task arrivals, we employ the Poisson distribution as a mathematical model in our simulation.

The Poisson distribution is a commonly used probability distribution that is suitable for describing the randomness of the number of events occurring within a fixed time interval. For the computational task quantity $A_n(t)$ generated by MSD_n in each time slot t , we represent it using the following Poisson distribution formula:

$$P(A_n(t) = \mathcal{G}) = \frac{e^{-\lambda_n \cdot t} \cdot (\lambda_n \cdot t)^{\mathcal{G}}}{\mathcal{G}!} \quad (1)$$

where λ_n is the average arrival rate of computational tasks for MSD_n within each time slot t , and $\mathcal{G}$ represents the number

of tasks. This formula describes the probability of a specific number of tasks arriving within a given time slot.

To limit the upper bound of the task quantity, we can introduce A_n^{max} as the limit for the task quantity, i.e., $A_n(t) \leq A_n^{max}$. By appropriately setting the parameter λ_n in the Poisson distribution, we can control the average arrival rate of tasks and use A_n^{max} to restrict the maximum number of tasks.

B. Local Execution Model

MSDs themselves have certain data processing capabilities and it is possible to consider offloading data for local processing. Thus, we consider the processing capability and power management of each device MSD_n at different time slots t . The computational capacity of device MSD_n at time slot t is denoted as $r_n(t)$ (cycles/bit), which represents the device's computational efficiency in processing incoming tasks. Additionally, the device can manage its power consumption by adjusting the CPU cycle frequency $f_n^L(t)$ (cycles/s), where $f_n^L(t)$ must not exceed the device's maximum computing speed f_n^{max} to ensure that tasks can be processed at the maximum computing speed. The local latency can be represented as $T_n^L(t)$, which is calculated using the formula:

$$T_n^L(t) = \sum_{m=1}^M \frac{r_n(t)}{f_n^L(t)} \cdot [1 - C_n^m(t)] \cdot A_n(t) \quad (2)$$

where $C_n^m(t)$ represents the proportion of computational tasks offloaded from MSD_n to BEC_m at time slot t ($n \in \mathcal{N}$, $m \in \mathcal{M}$). The local execution power can be represented as $p_n^L(t)$, with the following formula:

$$p_n^L(t) = k_n^L \cdot [f_n^L(t)]^3 \quad (3)$$

where k_n^L refers to the constant power coefficient. It should be noted that the allocation of computational and wireless resources to MSD remains unchanged from one time slot to the next in the system. Accordingly, the local energy consumption can be represented as $E_n^L(t)$, with the following formula for its calculation:

$$E_n^L(n) = p_n^L(t) \cdot T_n^L(t) = k_n^L \cdot T_n^L \cdot [f_n^L(t)]^3 \quad (4)$$

C. Computation Model of Edge Server

In the data offloading scenario for the BECIN architecture depicted in Fig. 1, we incorporate multiple edge computing servers based on Blockchain technology. For each time slot t , the computing tasks generated by each MSD device MSD_n can be offloaded to different BEC servers BEC_m , with the offloading proportion represented by $C_n^m(t)$. Each BEC server BEC_m possesses a specific quantity of computing resources f_m and bandwidth resources B_m . Furthermore, the BEC system supports dynamic adjustment of the CPU cycle frequency to reduce power consumption.

To allocate computing resources based on task requirements, we define the overall computing resource allocation as $P_{c,m}(t)$.

To facilitate resource allocation, we introduce the resource allocation coefficient $G_n^m(t)$, which represents the resource allocation from BEC server BEC_m to MSD device MSD_n at time slot t , subject to the constraint $G_n^m(t) \in P_{c,m}(t)$.

The allocation of computing resources (e.g., CPU cycle frequency) from edge server BEC_m to MSD_n at time slot t can be expressed as:

$$f_n^{m,t}(P_{c,m}^t) = f_m \cdot \frac{G_n^m(t)}{\sum_{i \in \mathcal{N}_m^t} G_i^m(t)}, n \in \mathcal{N}_m^t, t \in \mathcal{T} \quad (5)$$

Based on the observation, when $G_i^m(t) = G_j^m(t), \forall i \neq j$, the edge server BEC_m evenly distributes the computational resources among the available MSDs. $\mathcal{N}_m^t$ refers to the set of MSDs within the wireless coverage of BEC_m at time slot t . The total computation task latency of BEC_m at time slot t is:

$$T_m^{E,t}(P_{c,m}^t) = \sum_{i \in \mathcal{N}_m^t} \frac{\Psi_m(t)}{f_m^{i,t}(P_{c,m}^t)} \cdot C_i^m(t) \cdot A_i(t), m \in \mathcal{M}, t \in \mathcal{T} \quad (6)$$

where $\Psi_m(t)$ represents the computational capability of BEC_m in processing incoming tasks, measured in cycles per second. Additionally, the energy consumption of BEC_m at time slot t is given by:

$$\begin{aligned} E_m^{E,t}(P_{c,m}^t) &= K_m^E \cdot [f_m^{i,t}(P_{c,m}^t)]^3 \cdot T_m^{E,t}(P_{c,m}^t) \\ &= K_m^E \cdot \sum_{i \in \mathcal{N}_m^t} \left\{ \Psi_m(t) \cdot [f_m^{i,t}(P_{c,m}^t)]^2 \cdot C_i^m(t) \cdot A_i(t) \right\} \end{aligned} \quad (7)$$

where K_m^E denotes the BEC energy consumption for executing 1 CPU/GPU cycles (expected CPU/GPU cycles required by a task) and is set as 9×10^5 Wh.

D. Communication Model

In communication models, the communication between MSDs and BECs is established based on Orthogonal Frequency Division Multiple Access (OFDMA), which serves as a widely adopted multiple access scheme. OFDMA partitions the total bandwidth resources of the edge server BEC_m into K equal orthogonal subchannels, each with a bandwidth of w_m Hz, to support communication between MSDs. The set of subchannels used for MSD communication is denoted as $\mathcal{K} = \{1, \dots, k\}$. It should be emphasized that each subchannel can be allocated to a maximum of one terminal device. Such a design enables simultaneous communication of multiple terminal devices on different subchannels, thereby enhancing the overall system capacity and efficiency. By dividing the available spectrum into orthogonal subchannels, different terminal devices can transmit data in parallel during the same time interval, facilitating efficient utilization of the spectrum resources. Additionally, the adoption of orthogonal subchannel allocation helps mitigate interference issues as each terminal device exclusively occupies a subchannel, avoiding mutual interference between devices.

At time slot t , the allocated bandwidth from edge server BEC_m to MSD_n can be represented as follows:

$$W_n^{m,t}(P_{b,m}^t) = B_m \cdot \frac{X_{n,k}^{m,t}}{\sum_{i \in \mathcal{N}_m^t} X_{i,k}^{m,t}}, n \in \mathcal{N}_m^t \quad (8)$$

here, $X_{i,k}^{j,t} \in \{0, 1\}, i \in \mathcal{N}_j^t, j \in \mathcal{M}$. When $X_{i,k}^{j,t} = 1$, it indicates that MSD_n is associated with subchannel k for data transmission to BEC_m . Conversely, when $X_{i,k}^{j,t} = 0$, it indicates no association. In order to allocate computational resources based on task requirements, we define the overall communication bandwidth resource allocation as follows:

$$P(t) = \left\{ X_{i,k}^{j,t} \mid i \in \mathcal{N}_j^t, j \in \mathcal{M}, k \in \mathcal{K}, t \in \mathcal{T} \right\} \quad (9)$$

In each time slot, bandwidth resources are allocated based on demand, with the total bandwidth allocation denoted as $P_{b,m}^t, m \in \mathcal{M}$. $X_{n,k}^{m,t}$ represents the bandwidth allocation coefficient from BEC server BEC_m to MSD device MSD_n at time slot t , subject to the constraint $X_{n,k}^{m,t} \in P_{b,m}^t$. Additionally, we observe that in (8), when $n \in \mathcal{N}_m^t$ and $\forall X_{n,k}^{m,t} = 1$, the communication resources of BEC_m are equally distributed among MSD_n .

In OFDMA systems, each user is assigned a separate subcarrier for data transmission, which ensures orthogonality among users and effectively eliminates inter-cell interference—that is, interference between different users or devices using the same subcarrier across adjacent cells. This separation in the frequency domain not only simplifies resource allocation but also significantly improves system capacity and reduces interference. However, intra-cell interference can still arise within a single cell. This occurs when multiple transmitters, such as base stations or edge computing nodes, reuse the same subcarrier. Such interference may be caused by factors like multipath propagation or uncoordinated transmissions, and if not properly managed, it can impact system performance. To address this, effective scheduling and interference management strategies are essential [41].

In our system, we use OFDMA and allow each mobile smart device to occupy only one subchannel at a time, reducing the chance of intra-cell interference under the assumption of orthogonal allocation. However, at any given time slot t , a task from one MSD can be offloaded to multiple BEC nodes, which may reuse the same subchannel. This leads to inter-cell interference, defined here as interference between different cells' BEC nodes transmitting over the same subcarrier simultaneously. To quantify this, when BEC_m transmits to MSD_n at time slot t , the level of inter-cell interference can be calculated as follows:

$$\Upsilon_{n,k}^{m,t}(P_{b,m}^t) = \sum_{j \neq m} \sum_{i \in \mathcal{N}_j^t} X_{i,k}^{j,t} \cdot P_i^t \cdot H_{i,k}^{j,t} \quad (10)$$

where $\sum_{j \neq m}$ refers to the set of BECs except BEC_m , $\sum_{i \in \mathcal{N}_j^t}$ refers to the set of MSDs outside the coverage area of BEC_m , P_i^t represents the transmission power of MSD_i , and $H_{i,k}^{j,t}$ refers to the channel power gain from BEC_j , divided by subchannel k , to MSD_i . The complete formula for $H_{i,k}^{j,t}$ is given below:

$$H_{i,k}^{j,t} = \varepsilon \cdot l_{i,k}^{j,t} - \vartheta \cdot h_{i,k}^{j,t} \quad (11)$$

where ε represents the coefficient of path loss, and ϑ represents the exponent of path loss. $h_{i,k}^{j,t}$ represents the channel power gain resulting from small-scale fading. $l_i^{j,t}$ indicates the distance between the location l_i^t of MSD_i and the location l_j of BEC_j at time slot t . The detailed expression of the formula is as follows:

$$l_i^{j,t} = \|l_j - l_i^t\| \quad (12)$$

here, $\|\cdot\|$ refers to the vector norm, which represents the length of a directed line segment in two-dimensional space.

During the process of communication, signals are affected by various interferences and noises, such as interference signals from other users, multipath interference resulting from multipath propagation, and environmental noise. These interferences and noises significantly deteriorate the quality of the received signal, thereby impacting the overall performance of the communication system. To assess the quality of the signal, the Signal-to-Interference-plus-Noise Ratio (SINR) is employed as a metric. It quantifies the ratio of the received signal power to the combined power of interference and noise.

Hence, building upon the aforementioned context, the mathematical expression for the SINR from MSD_n to BEC_m on subchannel k at time slot t can be represented as follows:

$$\Psi_{n,k}^{m,t}(P_{b,m}^t) = \frac{P_n^t \cdot H_{n,k}^{m,t}}{\Upsilon_{n,k}^{m,t}(P_{b,m}^t) + \sigma^2}, n \in \mathcal{N}_m^t, t \in \mathcal{T}, k \in \mathcal{K} \quad (13)$$

In the context provided, where P_n^t represents the transmission power of MSD_n and σ^2 corresponds to the noise power spectral density, the SINR serves as a pivotal parameter for estimating the channel's quality and subsequently calculating the transmission rate. The computation of the transmission rate predominantly revolves around determining the channel capacity, which signifies the maximum achievable data rate given the channel conditions. In an ideal scenario free from interference, the channel capacity can be derived by applying Shannon's formula: $C = B \cdot \log_2(1 + SINR)$. Here, C symbolizes the channel capacity, whereas B represents the channel bandwidth. The logarithm with a base of 2, denoted as $\log_2$, encapsulates the relationship. Thus, during time slot t , the transmission rate amid BEC_m and MSD_n can be mathematically expressed as follows:

$$r_n^{m,t}(P_{b,m}^t) = W_n^{m,t}(P_{b,m}^t) \cdot \log_2(1 + \Psi_{n,k}^{m,t}(P_{b,m}^t)), \quad n \in \mathcal{N}_m^t, t \in \mathcal{T} \quad (14)$$

Therefore, based on the aforementioned transmission rate, we can compute the transmission delay from MSD_n to BEC_m . The mathematical expression is as follows:

$$T_{trans}^{m,n,t}(P_{b,m}^t) = \frac{C_n^{m,t} \cdot A_n(t)}{r_n^{m,t}(P_{b,m}^t)}, n \in \mathcal{N}_m^t, t \in \mathcal{T} \quad (15)$$

The energy consumption for the transmission from MSD_n to BEC_m at time slot t is defined as follows:

$$E_{trans}^{m,n,t}(P_{b,m}^t) = \frac{C_n^{m,t} \cdot A_n(t) \cdot P_n^t}{r_n^{m,t}(P_{b,m}^t)}, n \in \mathcal{N}_m^t, t \in \mathcal{T} \quad (16)$$

III. PROBLEM IDENTIFICATION AND ANALYSIS

In this section, we first analyze the constraints of the dynamic task offloading and resource allocation problem. Moreover, we present the formulation problem to achieve dynamic task offloading and resource allocation, aiming to minimize the weighted sum of latency and energy consumption.

A. Constraints of the Problem

1) *Offloading decision constraints*: Let $C_n^{m,t}$ represent the offloading decision of MSD_n at time slot t . When $\sum_{m=1}^M C_n^{m,t} = 0$ or 1, it respectively indicates that our task is fully allocated to the local MSD processor or the edge processor. Furthermore, we assume that each task is divisible, specifically, when $\sum_{m=1}^M C_n^{m,t} \in (0, 1)$, it signifies the task being allocated to multiple processors (local MSD processor and edge processor) for processing. Clearly, the task offloading constraints need to satisfy the following conditions:

$$\sum_{m=1}^M C_n^{m,t} \in [0, 1], t \in \mathcal{T} \quad (17)$$

2) *Local processor CPU cycle frequency constraint*: $f_n^L(t)$ represents the CPU cycle frequency of the local processor, which corresponds to the processing speed of the local processor. Specifically, the local processor can dynamically adjust the CPU cycle frequency to reduce power consumption. The local processor can only handle one task at a time, and its computational resources are limited. The computational configuration needs to satisfy the following conditions:

$$0 < f_n^{L,t} \leq f_n^{max}, \forall n \in \mathcal{N}, t \in \mathcal{T} \quad (18)$$

3) *Computational resource constraint of edge processors*: $f_n^{m,t}(P_{c,m}^t)$ refers to the computational resources allocated to MSD_n within the coverage range of BEC_m . Unlike the local processor, the edge processor can handle multiple tasks. Similarly, the computational resources of the edge processor are limited. It is evident that the following constraint needs to be satisfied:

$$\sum_{n \in \mathcal{N}_m^t} f_n^{m,t}(P_{c,m}^t) \leq f_m, \forall t \in \mathcal{T} \quad (19)$$

4) *Bandwidth constraint*: $w_n^{m,t}(P_{c,m}^t)$ refers to the communication resources allocated to MSD_n within the coverage range of BEC_m . The communication bandwidth resources of the edge server are limited. Therefore, it is necessary to satisfy the following constraint:

$$\sum_{n \in \mathcal{N}_m^t} w_n^{m,t}(P_{c,m}^t) \leq B_m, \forall t \in \mathcal{T} \quad (20)$$

5) *Wireless channel power constraint*: $P_n(t)$ refers to the wireless channel power of MSD_n at time slot t . P_n^{max} represents the maximum allowed wireless channel power for MSD_n during wireless communication at time slot t . This constraint is imposed to limit the wireless channel power of each MSD at a specific time slot, in order to avoid interference with other devices or exceed the system's power limits. The constraint can be formally

expressed as follows:

$$0 \leq P_n(t) \leq P_n^{max}, \forall n \in \mathcal{N} \quad (21)$$

6) *Subchannel constraint*: $X_{n,k}^{m,t}$ represents the bandwidth allocation coefficient from BEC server BEC_m to MSD device MSD_n at time slot t , subject to the constraint $X_{n,k}^{m,t} \in P_{b,m}^t$. Each subchannel is allocated to only one MSD, thus it is necessary to satisfy the following constraint:

$$\sum_{i \in \mathcal{N}} X_{i,k}^{j,t} \leq 1, \forall t \in \mathcal{T}, j \in \mathcal{M}, k \in \mathcal{K} \quad (22)$$

In addition, each MSD is allocated to a single subchannel, and it is evident that the following condition is satisfied:

$$\sum_{j \in \mathcal{M}} \sum_{k \in \mathcal{K}} X_{i,k}^{j,t} \leq 1, \forall t \in \mathcal{T}, i \in \mathcal{N} \quad (23)$$

B. Formulation of System Problem

Taking into account the constraints discussed in Section II-A, we propose a joint task offloading and resource allocation problem in dynamic environments.

In this problem, we need to simultaneously consider strategies for task offloading and resource allocation to optimize system performance. In a constantly changing environment, we need to dynamically determine (1) the tasks, which should be offloaded to edge servers for processing and (2) how to allocate limited computing and communication resources to maximize system efficiency.

In the system, we define the total cost as the weighted sum of task delay and processing energy consumption. Task latency encompasses both transmission latency and computation latency. For a given time slot t , the total latency of task transmission and computation on local processor MSD_n and edge server BEC_m can be expressed as:

$$T(m, n, t) = T_n^{L,t} + T_m^{E,t}(P_{c,m}^t) + T_{trans}^{m,n,t}(P_{b,m}^t) \quad (24)$$

For a given time slot t , the total energy consumption of task transmission and computation on local processor MSD_n and edge server BEC_m can be expressed as:

$$E(m, n, t) = E_n^{L,t} + E_m^{E,t}(P_{c,m}^t) + E_{trans}^{m,n,t}(P_{b,m}^t) \quad (25)$$

Based on the task latency as shown in (24) and the energy consumption as shown in (25), we define the following optimization objective:

$$Q(m, n, t) = \phi \cdot T(m, n, t) + (1 - \phi) \cdot E(m, n, t) \quad (26)$$

Thus, the BECIN problem is formulated as:

-p:

$$\min_{C_n^{m,t}, P_{c,m}^t, P_{b,m}^t} C_{total} = \frac{1}{T} \sum_{t=0}^{T-1} \sum_{m=1}^M \sum_{n \in \mathcal{N}_m^t} Q(m, n, t) \quad (27)$$

-subject to: [17—23]

In light of the BECIN optimization problem, the presence of a time-varying system state, encompassing the stochasticity of task arrivals and the potential variability in resource availability, poses challenges for traditional optimization methods,

such as Lyapunov optimization, to directly tackle the specified problem [42]. To address these challenges, we develop a pioneering deep reinforcement learning (DRL) algorithm capable of effectively capturing the dynamic characteristics of the system and seamlessly integrating them into the solution's design. This algorithm is specifically designed to adaptively address the time-varying nature of the system state through the learning process, utilizing the acquired knowledge to optimize the solution to the problem at hand.

IV. PROPOSED SCHEME

In this paper, we propose a joining and exiting mechanism based on the combination of blockchain and cryptography to address the problem of updating service policies in BECIN according to dynamic task arrivals and resource changes. Additionally, we introduce a state-of-the-art DRL algorithm called Dynamic Dueling DDQN to approximate optimal task offloading and resource allocation strategies. The overall process is illustrated in Fig. 2.

A. Joining and Exiting Mechanism

Our proposed approach aims to tackle the challenge of adapting service policies in BECIN based on the dynamic nature of task arrivals and resource fluctuations. We leverage the immutability and decentralization properties of blockchain technology to facilitate the dynamic joining and exiting of edge service providers. When an edge service provider joins, its relevant information is recorded on the blockchain, allowing other participants to access and update their service policies accordingly. Similarly, when an edge service provider exits, its information is removed from the blockchain, ensuring that the system only includes valid service providers.

To increase the flexibility of the protocol, we introduce the Join-and-Exit mechanism. The entire process is coordinated by a smart contract.

Join: If a new BEC_m wants to participate in our protocol, it can invoke the Join function defined in the smart contract. During this process, the node first submits a registration request and initiates identity verification using cryptographic credentials. Once the verification is successful, the smart contract automatically updates the blockchain ledger and grants access to the new node. We implement an auditable secure multi-party computation for the BEC nodes based on the Chinese Remainder Theorem (CRT) verifiable threshold secret sharing scheme. Additionally, we adopt an improved CRT Paillier scheme to ensure secure key transmission among the BEC nodes during the auditing process. The detailed steps are as follows.

- **System Initialization**: Let D be the registration authority of the BEC system, responsible for system registration and exit management, including key generation, secret distribution, and data deletion upon exit. BEC_m (where $m \in \mathcal{M}$) represents the currently joined M participants in the system, with a threshold value set to t_b and a shared secret set as S_b . The threshold value functions similarly to the consensus mechanism in blockchain nodes, where a new node can only join the network after completing

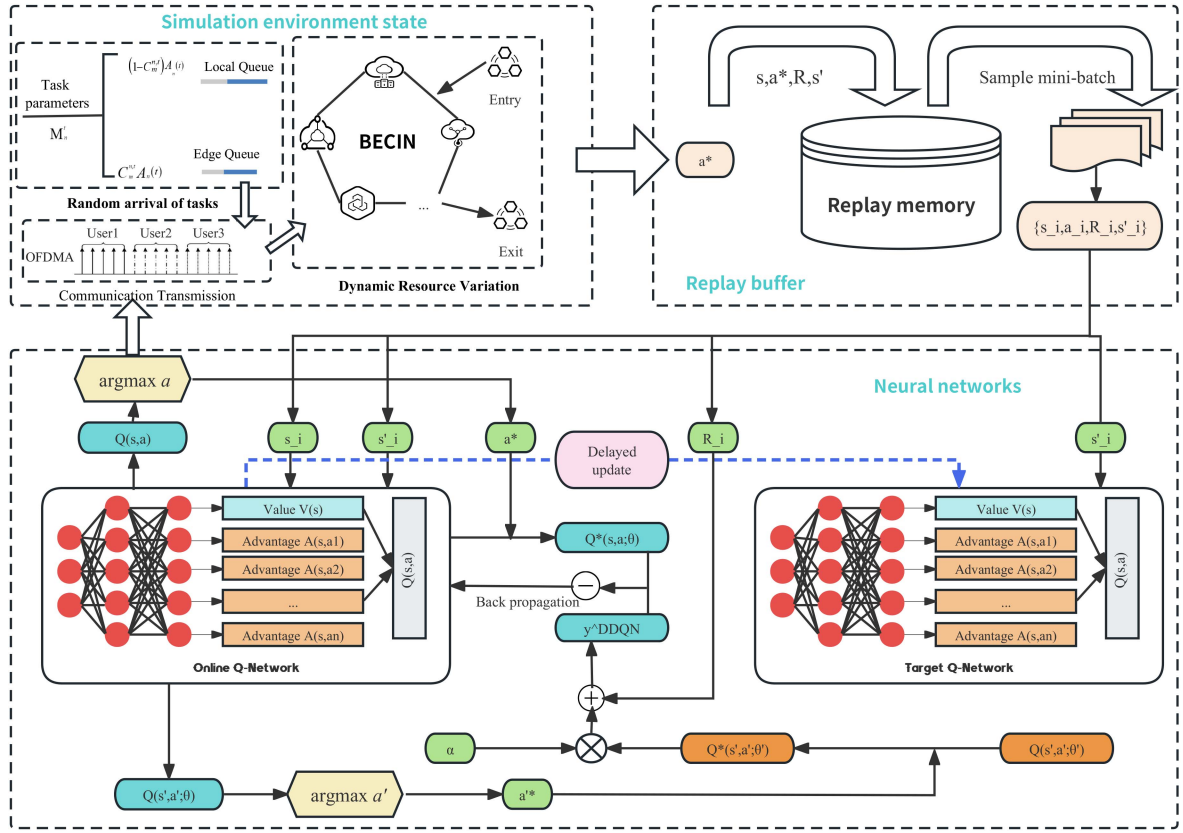


Fig. 2. Dynamically self-adjusting BECIN framework based on Dueling DDQN.

the computation and obtaining verification from a specific number of other nodes.

D selects a large prime number p_d and g as an element in the finite field Z_N , where N is an integer satisfying the hardness of integer factorization, and p_d is greater than the integer S_b . g_i is an element of order m_i in Z_{p_i} . Additionally, a positive integer sequence $l_d = \{l_1, l_2, \dots, l_M\}$ is selected, satisfying the following conditions: (1) $l_1, l_2, \dots, l_M$ are strictly increasing; (2) $(l_i, l_j) = 1$, ($i \neq j$); (3) $(l_i, p) = 1$, ($i = 1, 2, \dots, M$); (4) $N = \prod_{i=1}^t l_i > p_d \prod_{i=1}^{t-1} l_{M-i+1}$; (5) ensuring $p_i = 2 \cdot l_i + 1$ remains prime, ($i = 1, 2, \dots, M$). Let $M_b = \prod_{i=1}^t l_i$, and select a positive integer A satisfying $0 < A < M_b$, where $y = d + A \cdot p_d < M_b$. D publicly discloses p_d , A , l_d , and g to all BEC nodes.

- **Generation of Secret Shares and Verification Information:** The secret shares in this scheme are generated jointly by the system registration authority D and M BEC nodes. Firstly, D calculates $X = S_b + A \cdot p_d$ and randomly decomposes X into M non-zero positive integers $X_1, X_2, \dots, X_M$, such that $X = X_1 + X_2 + \dots + X_M$. Then, X_i is sent to each BEC node separately ($i = 1, 2, \dots, M$). Upon receiving X_i , each BEC node BEC_i utilizes its secret data X_i to compute $a_{ij} = X_i \bmod l_j$, where $j = 1, 2, \dots, M$. The value a_{ii} is kept secret, while a_{ij} ($j \neq i$) is secretly transmitted to other BEC_j . Additionally, each BEC_i calculates its verification information, denoted as α_i and β_{ij} ,

as follows:

$$\alpha_i = g^{X_i} \bmod p_d \quad (28)$$

$$k_{ij} = (X_i - a_{ij})/l_j \quad (29)$$

$$\beta_{ij} = g^{k_{ij}} \bmod p_d (j = 1, 2, \dots, M) \quad (30)$$

After calculating their respective verification information, α_i and β_{ij} are publicly disclosed. Finally, it is known that after completing the aforementioned process, the secret share received by BEC_i ($i = 1, 2, \dots, M$) is $S_i = \{a_{ji} | j = 1, 2, \dots, M\}$.

- **Validation of Secret Shares:** In the previous system initialization phase, we mentioned that the threshold value is set to t_b , which means that at least t_b nodes need to approve for a new node to join. If M BEC nodes express affirmation towards the newly added BEC node, it indicates their willingness to share their secret shares with other consenting parties. If the number of sharable secret shares is less than t_b , D notifies the relevant prospective edge service providers, informing them that joining the blockchain network is not allowed. Let the t_b BEC nodes be $\{BEC_1, BEC_2, \dots, BEC_{t_b}\}$. To prevent dishonesty among BEC nodes in the blockchain network, BEC_k , upon receiving secret shares S_j from other BEC_j , verifies n components a_{ij} using the following validation formula

($i = 1, 2, \dots, M$):

$$\lfloor (g^{a_{ij}} \bmod p_d) (\beta_{ij}^{m_j} \bmod p_d) \rfloor \bmod p_d = \alpha_i \quad (31)$$

If (31) holds, the share S_j is considered valid; otherwise, S_j is deemed invalid, indicating that BEC_j has provided a false share.

- *Secret Recovery*: After the verification of the t_b secret shares mentioned above, the system registration authority D securely sends these secret shares to any participant. Each participant receives t_b secret shares as follows: $S_1 = a_{11}, a_{21}, \dots, a_{n1}, S_2 = a_{12}, a_{22}, \dots, a_{n2}, \dots, S_{t_b} = a_{1t_b}, a_{2t_b}, \dots, a_{nt_b}$. Each participant constructs M sets of congruence equations based on these t_b secret shares, as follows:

$$\begin{cases} X_1 \equiv a_{11} \bmod l_1 \\ X_1 \equiv a_{12} \bmod l_2 \\ \dots \\ X_1 \equiv a_{1t_b} \bmod l_{t_b} \end{cases}, \quad \begin{cases} X_2 \equiv a_{21} \bmod l_1 \\ X_2 \equiv a_{22} \bmod l_2 \\ \dots \\ X_2 \equiv a_{2t_b} \bmod l_{t_b} \end{cases}, \quad \vdots \quad \begin{cases} X_M \equiv a_{M1} \bmod l_1 \\ X_M \equiv a_{M2} \bmod l_2 \\ \dots \\ X_M \equiv a_{Mt_b} \bmod l_{t_b} \end{cases} \quad (32)$$

According to the CRT, solving these M systems of equations results in unique values for $X_1, X_2, \dots, X_M$. By calculating $X = X_1 + X_2 + \dots + X_M$ and using the formula $X = S_b + A \cdot p_d$, the shared secret S_b can be obtained as $S_b = X - A \cdot p_d$.

Obtaining the shared secret S_b implies the attainment of qualification for joining the BEC network, signifying a successful registration.

Exit: If an existing BEC_m requests to exit the protocol, the process is similar to the join process. The node constructs a transaction to trigger the smart contract's Exit function, which verifies its identity and proceeds to revoke access. All associated metadata stored on-chain is then removed or marked inactive, ensuring only valid nodes remain in the network. Then, we introduce the related theorems and their proofs of our proposed framework.

Theorem 1: The join and exit mechanism can verify user identity to ensure the security of the signature authentication scheme, as long as: $\lfloor (g^{a_{ij}} \bmod p_d) (\beta_{ij}^{l_j} \bmod p_d) \rfloor \bmod p_d = \alpha_i$.

Proof: See Appendix I.A.

Theorem 2: Participants can recover the secret as long as they possess at least t_b secret shares.

Proof: See Appendix I.B.

Theorem 3: It can prevent deception behavior from the system secret distributor D .

Proof: See Appendix I.C.

Theorem 4: It is capable of identifying mutual deception behavior among the M BEC nodes in the BEC network.

Proof: See Appendix I.D.

Theorem 5: The verification process is secure.

Proof: See Appendix I.E.

B. Proposed Dynamic Dueling DDQN

To achieve optimal task offloading and resource allocation strategies, we employ the Dueling DDQN algorithm, which trains a dual Q-network to approximate the optimal policy with two Q-networks competing against each other to enhance the stability and convergence speed of the policy. By utilizing the dueling DDQN algorithm, we can dynamically update task offloading and resource allocation strategies in response to real-time task arrivals and resource changes in the dynamic environment.

Then, we provide a comprehensive account of the origins of Dueling DDQN, including an introduction to its network architecture. Subsequently, to effectively present our framework, we incorporate a detailed description of our specific settings within the given context, encompassing *reward*, *action*, and *state*, among other elements.

The Origins of Dueling DDQN: Traditional reinforcement learning algorithms commonly adopt tabular representations for storing state value functions $V(s)$ or state-action value functions $Q(s, a)$. However, this approach exhibits some limitations. For instance, real-world reinforcement learning tasks frequently entail continuous state spaces characterized by an infinite number of states, thereby rendering tabular storage infeasible. To overcome the aforementioned limitations, value function approximation has emerged as a solution by directly fitting a function to approximate the state value function or the state-action value function. This approach reduces storage requirements and effectively addresses the problem.

In the context of continuous state spaces, the value function $Q^\pi(s, a)$ can be computed by employing an approximation function denoted as $Q_\phi(s, a)$. Mathematically, we have:

$$Q_\phi(s, a) \approx Q^\pi(s, a) \quad (33)$$

Here, s and a represent the vector representations of the state s and action a , respectively. The function $Q_\phi(s, a)$ is typically parameterized with ϕ , often implemented as a neural network, which outputs a real value and is commonly referred to as the Q-network.

The core of the Deep Q-Network (DQN) algorithm lies in maintaining the Q function and utilizing it for decision-making. $Q^\pi(s, a)$ denotes the action-value function under policy π . After reaching state s_t , the algorithm traverses the entire action space, selecting the action that maximizes $Q^\pi(s, a)$ as the policy:

$$a_t = \arg \max_a Q^\pi(s_t, a) \quad (34)$$

Then, DQN employs the Bellman equation to update $Q^\pi(s, a)$:

$$Q^\pi(s_t, a_t) \leftarrow Q^\pi(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q^\pi(s_{t+1}, a) - Q^\pi(s_t, a_t) \right) \quad (35)$$

In simpler tasks, fully connected neural networks are often used to approximate Q^π . However, for more complex tasks like playing Atari games, convolutional neural networks are employed to approximate the mapping from images to value functions. Due to its limited capability in handling a finite number of action values, DQN is commonly employed in tasks with discrete action spaces. In the original DQN algorithm, when updating the Q-values using the Bellman equation, the maximum action value at the current state is used to estimate the target Q-value. However, this estimation can introduce a high bias, leading to unstable algorithm performance. Double DQN proposes a correction to mitigate this bias. The core idea of Double DQN is to separate the computation of the target Q-value and the action selection processes. In the computation of the target Q-value, the current Q-network is first used to select the optimal action a_{t+1} . Then, the target Q-network is used to estimate the Q-value $Q^\pi(s_{t+1}, a_{t+1})$ corresponding to this action. As a result, the computation formula for the target Q-value becomes:

$$Q^{\text{target}}(s_t, a_t) = r_t + \gamma Q^\pi \left(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta) \right) \quad (36)$$

where θ represents the parameters of the target Q-network. The purpose of this approach is to avoid overestimating the current Q-value during the computation of the target Q-value. During the training process, the target Q-value is computed using the target Q-network, while the estimated Q-value is computed using the current Q-network. Then, the difference between these two values is calculated using mean squared error, and the parameters of the current Q-network are updated accordingly.

The main improvement of Double DQN is aimed at reducing the overestimation of target Q-values, thereby enhancing the stability and performance of the algorithm. Dueling DQN is another method for improving DQN, which involves separating the state value function and the action advantage function. This separation allows for a better estimation of the relative advantages of different actions in each state, thereby enhancing the algorithm's performance and efficiency. In this paper, Dueling DDQN combines two improvement methods by utilizing the network structure of Dueling DQN and the target Q-value estimation approach of Double DQN to further enhance the algorithm's performance. Specifically, traditional DQN uses a single Q-network to output Q-values for each action, which does not fully utilize the information in the state. Dueling DQN introduces a network structure that separates the state value and action advantage to model the relationship between the state value function and the action value function more effectively.

The network structure of Dueling DQN consists of two sub-networks: the value function and the advantage function. The value function represents the value of a state, while the advantage function represents the relative advantages of selecting different actions given a certain state. Specifically, the calculation formula for the state value function is:

$$V(s; \theta, \beta) = \sum_a \pi(a|s; \beta) \left(Q(s, a; \theta) - \frac{1}{|A|} \sum_{a'} Q(s, a'; \theta) \right) \quad (37)$$

here, $\pi(a|s; \beta)$ represents the probability of selecting action a given state s , θ denotes the network parameters, and β represents the parameters of the state value function. Specifically, the calculation formula for the action advantage function is:

$$A(s, a; \theta, \alpha) = Q(s, a; \theta) - V(s; \theta, \beta) \quad (38)$$

where α is the parameter of the action advantage function.

Finally, the value function of a state and the action value function are computed by adding (37) and 38 together:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + A(s, a; \theta, \alpha) \quad (39)$$

Specifically, Dueling DDQN mainly includes the following parts:

1) *Network Structure*: Dueling DDQN employs the network structure of Dueling DQN, which consists of the value function and the advantage function. These two functions share some layers but are split into separate branches before the output layer. This allows the network to simultaneously learn the value of states and the relative quality of different actions. The value function is used to estimate the value of states, representing the expected cumulative reward given a specific state in (37). The action advantage function is used to estimate the relative superiority or inferiority of different actions, indicating the advantage or disadvantage of taking a particular action compared to the average level in (38).

2) *Estimation of the target Q-value*: In computing the target Q-value, Dueling DDQN utilizes the estimation approach of Double DQN. Specifically, when selecting the optimal action, the current Q-network is employed to determine the optimal action, denoted as a_{t+1} . Subsequently, the target Q-network is used to estimate the Q-value corresponding to this action, denoted as $Q^\pi(s_{t+1}, a_{t+1})$. The specific formula can be found in (36). This approach helps reduce the overestimation of the target Q-value and enhances the stability of the algorithm. The final estimation of the Q-value is achieved by combining the state-value function and the action advantage function, as illustrated in (39).

To better describe the proposed self-adjusting BECIN framework based on Dueling DDQN, we first define *reward*, *action*, and *state* of Dueling DDQN in our scenario.

Reward: Here, our system's reward stems from the joint optimization of task latency and power consumption. Our objective is to maximize system reward over the long term while minimizing system latency and energy consumption. Therefore, a multiplication operation of -1 is performed on the reward before proceeding. In other words, we set the reward as $r(s_t, a_t) = -Q(m, n, t)$. To present the reward more effectively, we transform the problem of minimizing task latency and power consumption into a QoE optimization problem using an exponential function [43]:

$$QoE = \left[10^{1 - \frac{Q(m, n, t)}{\tau}} - 1 \right] \quad (40)$$

In this context, τ refers to an upper bound on the weighted sum of system latency and energy consumption. As indicated by (40), lower system latency and energy consumption result in higher QoE. Therefore, we focus on maximizing system QoE rather than minimizing latency and energy consumption to better

characterize the efficiency of our algorithm. Based on (40), we transform the optimization problem in (27) into:

-P:

$$\min_{C_n^{m,t}, P_{c,m}^t, P_{b,m}^t} Q_{total} = \frac{1}{T} \sum_{t=0}^{T-1} \sum_{m=1}^M \sum_{n \in \mathcal{N}_m^t} QoE \quad (41)$$

-subject to: [17—23]

Therefore, we set the reward as $r(s_t, a_t) = QoE$.

Action: To address environmental uncertainties and variations in task requirements, the system employs a dynamic action space mechanism. Specifically, the system assigns an independent neural network output layer (i.e., "action head") to each task, responsible for predicting the action decisions for that task. As the number of tasks changes, the system dynamically activates or deactivates the corresponding action heads, with each action head predicting the possible actions for its assigned task (e.g., offloading, bandwidth allocation, computation resource allocation, etc.). The system maintains an action head list with a maximum capacity of N , ensuring that the activation states of the action heads are adjusted in accordance with the changes in the number of tasks.

This design ensures that even with dynamic changes in the number of tasks, the system can generate independent and effective action decisions for each task. Additionally, by utilizing a shared feature extraction network layer, the system maintains coordination between tasks, thereby optimizing resource allocation and task scheduling. Based on this, the action space can be expressed as follows:

$$\begin{aligned} \mathcal{A}_t &= \{a_n^t | n \in \mathcal{N} = \{1, 2, \dots, N\}\} \\ &= \{(C_n^{m,t}, P_{b,m}^t, P_{c,m}^t) | n \in \mathcal{N} = \{1, 2, \dots, N\}\} \end{aligned} \quad (42)$$

where each action compoment is explained as follows:

- Offloading decision $C_n^{m,t}$: $0 \leq C_n^{m,t} \leq 1$, ($n \in \mathcal{N}, m \in \mathcal{M}$). $C_n^{m,t}$ refers to the computational tasks generated by MSD_n at time slot t , which, after making decisions, are allocated to BEC_m in a certain proportion. We discretize the values between 0 and 1 to distribute the tasks according to a specified ratio.
- System bandwidth allocation scheme $P_{b,m}^t$: $P_{b,m}^t$ refers to the set of computational tasks transmitted by MSD_n through subchannel k within the coverage area of all BEC_m . $P_{b,m}^t = \{P_{n,k}^{m,t} | n \in \mathcal{N}_m^t, m \in \mathcal{M}, k \in \mathcal{K}, t \in \mathcal{T}\}$. In mobile communication systems, subchannels are used to partition the spectrum resources into multiple independent frequency bands or frequency segments. Furthermore, in the frequency domain, different subchannels may be affected by varying small-scale fading components, depending on the signal propagation paths and environment. Hence, we establish multiple subchannels with different configurations, including small-scale fading components and interference, to cater to the needs of mobile communication services. Due to variations in propagation paths and the allocation of bandwidth resources, significantly different transmission rates are achieved.

- BEC resource allocation $P_{c,m}^t$: $P_{c,m}^t$ refers to the set of computing resources allocated by BEC_m to MSD_n at time slot t . $P_{c,m}^t = \{G_n^{m,t} | n \in \mathcal{N}_m^t, m \in \mathcal{M}, k \in \mathcal{K}, t \in \mathcal{T}\}$, $0 \leq G_n^{m,t} \leq 1$. Each edge server allocates a portion of its computing resources based on the current resource status and task status to execute tasks.

State: Similarly, in response to changes in the number of tasks or resources, the system is capable of automatically adjusting the dimensions and structure of the state vector. For each task, we record its current state, including task location, resource requirements, and inter-task relationships. As the number of tasks changes, the system automatically recalculates the dimensions of the state space and accordingly adjusts the input layer of the neural network.

In our environment, the system state consists of five components: task state $S_{task}(t)$, channel state $S_{channel}(t)$, bandwidth state $S_{band}(t)$, and available computing resource state $S_{comp}(t)$. Therefore, the system state is defined as follows:

$$S_t = \{S_{task}(t), S_{channel}(t), S_{band}(t), S_{comp}(t)\} \quad (43)$$

- $S_{task}(t)$: $S_{task}(t)$ encompasses the information regarding all data processing tasks performed by the MSDs within the time slot t . Specifically, $S_{task}(t)$ is defined as follows:

$$S_{task}(t) = \{M_n^t = (x_n^t, y_n^t, Date_i, A_n(t)) | n \in \mathcal{N}\} \quad (44)$$

Here, x_n^t and y_n^t represent the coordinates of the location where the task is initiated, $Date_i$ refers to the deadline of the task, $A_n(t)$ represents the amount of computational task generated by each MSD at time slot t (measured in bits), and it satisfies the condition $A_n(t) \leq A_n^{max}$.

- $S_{channel}(t)$: The term $S_{channel}(t)$ represents the current network state, specifically the remaining capacity after network channel allocation, which is defined as follows:

$$S_{channel}(t) = K - \sum_{i \in \mathcal{N}_m^t} \sum_{j \in \mathcal{M}} \sum_{k \in \mathcal{K}} X_{i,k}^{j,t} \quad (45)$$

- $S_{band}(t)$: The term $S_{band}(t)$ represents the remaining amount of bandwidth resources in the current system, which is defined as follows:

$$S_{band}(t) = \sum_{m \in \mathcal{M}} B_m - \sum_{n \in \mathcal{N}_m^t} \sum_{m \in \mathcal{M}} W_n^{m,t} (P_{b,m}^t) \quad (46)$$

- $S_{comp}(t)$: The term $S_{comp}(t)$ represents the state of the current computing resources, which is defined as:

$$S_{comp}(t) = \{(x_m^t, y_m^t, N_{avai}^t, G^{c,t}) | m \in \mathcal{M}, t \in \mathcal{T}\} \quad (47)$$

In the equation above, x_m^t and y_m^t denote the current coordinates (horizontal and vertical) of the BEC service. N_{avai}^t represents the total number of available edge processors that can be used. $G^{c,t}$ represents the remaining computing resources in the system, which is calculated as follows:

$$G^{c,t} = \sum_{m \in \mathcal{M}} f_m - \sum_{n \in \mathcal{N}_m^t} \sum_{m \in \mathcal{M}} f_m^{n,t} (P_{c,m}^t). \quad (48)$$

The Dueling DDQN algorithm consists of the following four stages: 1) We initialize network weights, replay memory, and exploration rate; 2) The agent chooses an action based on the state. The environment will jump to the state according to the action, and get instant rewards and store them in the experience pool; 3) We compute target Q-value and update online network weights θ by minimizing the loss; 4) We regularly update target network weights, monitor changes in the current system state, update the state representation function, and adjust the dynamic update strategy; 5) We repeat steps 2, 3, and 4 until the specified number of episodes N_{episodes} is reached.

V. EXPERIMENTAL EVALUATIONS

In this section, we perform extensive simulations to verify the performance of the proposed framework.

A. Data and Experiment Setup

In the simulation process, we collect data from sensors and store it in separate files within the simulated environment. These files are wirelessly transmitted to the simulated smartphone for local computation. We use the EdgeCloudSim tool to simulate data transmission and offloading to edge devices. Our dataset primarily consists of image data from users for diagnostic purposes. The tasks are divided into general tasks aimed at optimizing image display and more complex tasks involving AI image segmentation. Task allocation is the result of a series of decisions made by the system, which decides whether to offload the tasks to local or edge devices. In our experiment, we designed an edge computing framework integrated with blockchain technology. The experiment was deployed on a Lenovo Legion Y7000p with an edge computing platform, connecting to the blockchain platform Ganache via the MetaMask wallet and deploying smart contracts through the Remix tool. After the contract is deployed, it can be verified on Etherscan to ensure code consistency.

In our architecture, a region (with both x-axis and y-axis ranging from 0 to 500) consists of multiple BEC servers representing the number of nodes participating in the access blockchain network. Here, we consider the joining and leaving of blockchain nodes, and the number of BEC servers varies within a range of approximately 5 to 20. The number of mobile smart devices ranges from 50 to 100. In our simulation, each mobile device generates one task to be processed per time slot. These tasks have varying computation and bandwidth requirements based on algorithm complexity and task size. We generate task arrivals according to a Poisson distribution with a parameter value of 5. The randomly generated data block sizes are chosen from the range of [10, 100] MB. Additionally, the computational capability of MSDs ranges from [40, 100] cycles/bit, and the computation resources of each MSD are randomly allocated from {0.5, 0.8, 1.0} GHz. The constant power coefficient of MSDs is set to $10 \cdot 10^{-27} \text{ Watt} \cdot [\text{s}^3/\text{cycles}^3]$. The transmission power of MSDs is set to 100mW. In our BECIN architecture, each BEC has a total bandwidth resource of 40MHz and a total computation resource of 10GHz to facilitate communication among devices in the region and task processing. The noise spectral density σ^2 is set to 10^{-9} W . For more details on related parameter

settings, please refer to Part II “Related Parameter Settings” in the appendix.

B. Compared Algorithms

To demonstrate the effectiveness of the proposed framework, we design experiments from the following aspects:

- 1) Our proposed solution is an algorithm based on dynamic adjustment, enabling it to learn and optimize its performance based on changes in the environmental state. Compared to other algorithms that lack dynamic adjustment capabilities, proposed solution is better suited to adapt to different environmental conditions and offer more flexible and stable performance.
- 2) When both our strategy and the baseline algorithm incorporate a dynamic self-adaptive adjustment mechanism, Dueling DDQN is capable of achieving results than the baseline methods.

In our environment, four classic algorithms, namely PPO [44], DDQN [45], DQN [46], and Random [47], have been chosen as baselines. The detailed information of each algorithm is provided below:

- *PPO (Proximal Policy Optimization)*: PPO is a policy optimization algorithm that iteratively optimizes policy parameters to perform reinforcement learning tasks. It utilizes a method called proximal policy optimization, which constrains the magnitude of policy updates to ensure the stability and convergence of the algorithm.
- *DDQN (Double Deep Q-Network)*: DDQN is a deep reinforcement learning algorithm that improves the classic Q-learning approach. It addresses the overestimation problem of Q-values by using two independent neural networks to improve the learning efficiency and stability.
- *DQN (Deep Q-Network)*: DQN is a deep learning-based reinforcement learning algorithm that combines deep neural networks with Q-learning. DQN takes the state as input, outputs Q-values for each action, and optimizes the policy by selecting actions based on the Q-values.
- *Random*: Random is a simple random selection algorithm that randomly selects actions with a uniform distribution at each decision point. Random algorithm does not consider the environment state or policy optimization.

PPO is a policy gradient-based method that offers good stability and convergence speed in large-scale tasks. However, its relatively high computational complexity makes it less effective in dynamic environments. DQN evaluates the value of states and actions using a single Q-function, but in dynamic environments, it is prone to Q-value overestimation, leading to instability. Double DQN alleviates this issue by using two networks, but it still estimates Q-values as a whole. Dueling Double DQN further improves upon this by using a Dueling architecture, which more effectively handles action selection and value estimation in high-dimensional state spaces, making it more stable and efficient in dynamic environments with frequent changes. Experimental results show that Dueling Double DQN significantly outperforms DQN, Double DQN, and PPO in dynamic environments, primarily due to its advantages in Q-value decomposition and

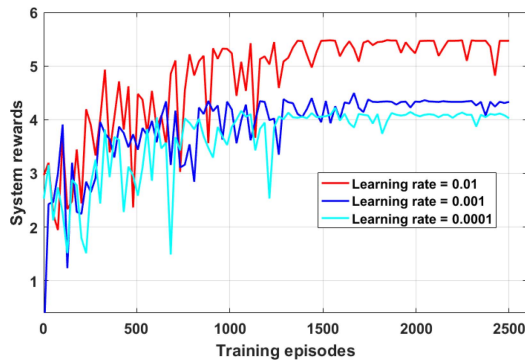


Fig. 3. Comparison of average system rewards with different learning rates.

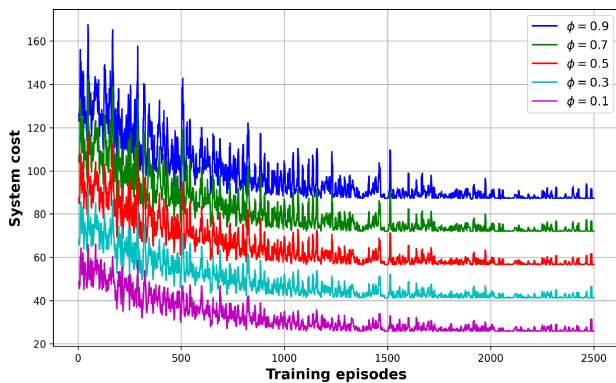


Fig. 4. The effect of the parameter ϕ on the system's Cost.

error correction, maintaining higher learning stability and lower training error.

C. Experimental Results

Here, we present the comparative results using different methods under different scenarios to verify the effectiveness of our proposed framework.

1) *Evaluation of Training Performance:* Fig. 3 illustrates the impact of different learning rates on the average system reward. A lower learning rate leads to a gradual increase in the system reward, relying on past experiences, reducing volatility but exhibiting slower convergence speed. Conversely, a higher learning rate results in a rapid increase in the system reward, being more sensitive to new observations, but potentially leading to unstable behavior. The selection of an appropriate learning rate is crucial, necessitating a trade-off between learning speed and stability. Experimental exploration and adjustments based on the specific problem and environment are required to achieve optimal performance. According to our experimental results, a learning rate of 0.01 yields the best reward performance with a satisfactory convergence rate. Therefore, we set learning rate as 0.01 for subsequent system simulations and evaluations. In addition, we have analyzed the effect of the parameter ϕ on the system's Cost in Fig. 4.

Fig. 5 demonstrates the impact of offloading data to all BEC servers on the average user expected cost and satisfaction. As the data offloading progresses, the average user expected cost

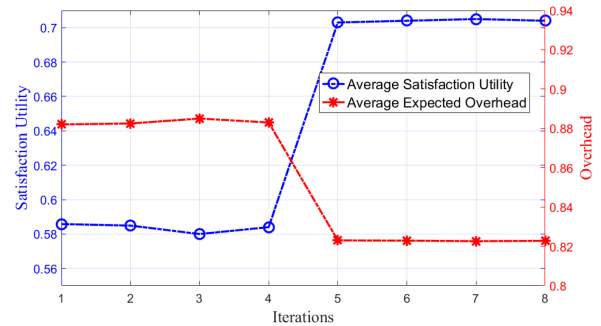


Fig. 5. Convergence of users' average expected overhead and satisfaction utility by offloading data to all the BEC servers.

gradually decreases, and satisfaction increases. The results illustrate that after the convergence to the optimal data offloading point, the users obtain high levels of satisfaction and low levels of expected overhead.

2) *Evaluation of RL Performance:* We evaluated the system cost and loss function performance of our proposed adaptive dynamic adjustment DRL approach and compared it with two other related approaches. The first approach is the traditional DRL method, which typically adopts static parameters or policies, meaning that they do not adjust during runtime based on changes in the environment. This static approach may lead to poor algorithm performance or instability in different environments. The second approach is the RL method without the use of neural networks. Typically, compared to DRL strategies, these methods rely on tabular representations of value functions or policies, which have limited modeling capabilities, especially in high-dimensional and continuous scenarios.

Fig. 6 illustrates the performance comparison of different RL approaches, showing significant improvements in lower cost consumption and lower losses compared to the baseline algorithm. Among them, (a)-(b) is a scene, and (c)-(d) is another scene. From the results, we observe that the Dynamic-based DRL consistently outperforms other algorithms. Furthermore, the cost expenditure and loss values exhibit a similar overall trend for the three algorithms (Dynamic-based DRL, Traditional DRL without Dynamic, Traditional RL), consistently decreasing and then stabilizing. This can be explained by the following reasons: (1) Dynamic mechanism: The Dynamic-based DRL architecture is capable of adapting to environmental changes by utilizing model-based methods to model the environment and perform online optimization for policy adjustments. By observing and analyzing the environment in real-time, appropriate actions are chosen in different states to maximize expected rewards and minimize unnecessary costs. (2) Learning capability and optimization process: Through the use of deep reinforcement learning methods, the policies are continuously learned and optimized by interacting with the environment, gradually improving and adapting to changes in various scenarios. In contrast, Traditional RL algorithms may perform poorly in nonlinear and highly dynamic environments, as they typically rely on manually extracted features and linear models. (3) Policy updates and stability: All three algorithms employ similar policy update mechanism, such as gradient-based methods, which

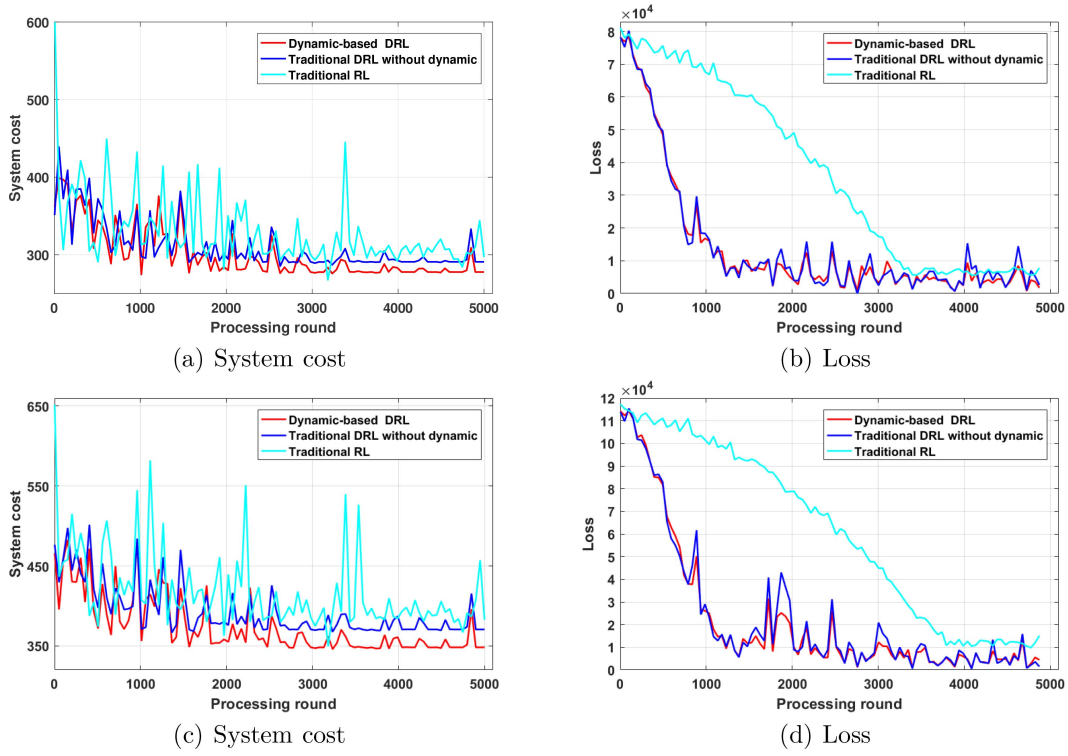


Fig. 6. Comparison of different RL approaches under the change of external resources.

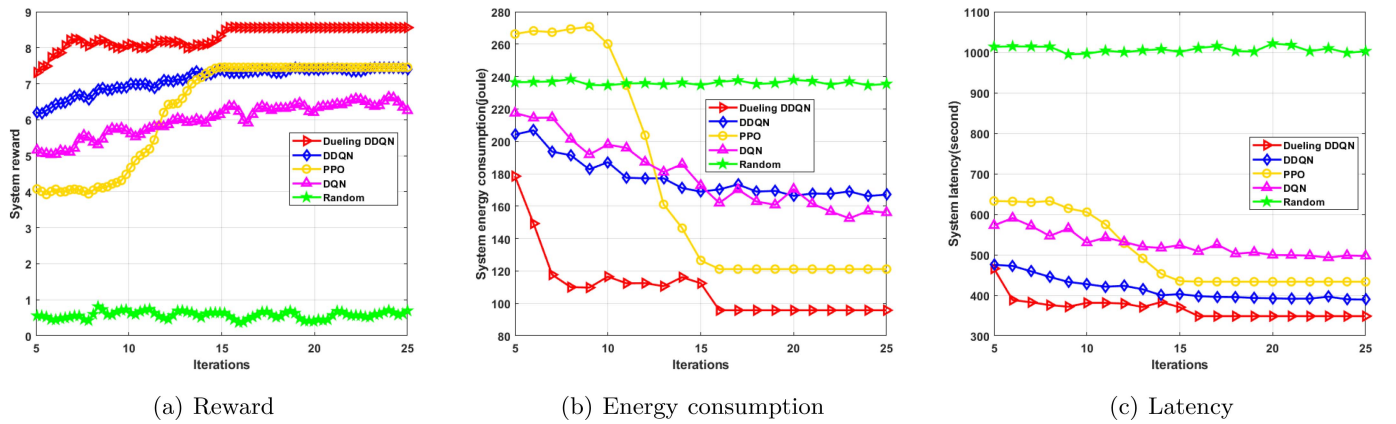


Fig. 7. The performance of different algorithms over iterations.

continuously adjust policy parameters for optimization. As the algorithms iterate and train, the policies converge and become stable, resulting in a decrease in cost expenditure and loss values.

3) *Evaluation of Task Offloading Utility: Performance comparison over the iterations:* Next, Fig. 7(a) illustrates the performance across different time steps and various system rewards. It can be observed that our Dueling DDQN consistently outperforms other algorithms in all scenarios. Additionally, the system rewards of the four algorithms (Dueling DDQN, DDQN, PPO, and DQN) exhibit a continuous increase followed by stabilization within the time steps, while Random shows relatively minor fluctuations. It can be seen that, when the number of iterations is small (≤ 15), the average system reward increases over the

iterations. This is because, at the beginning of the training, the agent may be in an initial random policy state with limited understanding of the environment, resulting in suboptimal action selection. As the number of time steps increases, the agent gradually learns from past experiences and optimizes its strategy, leading to an improvement in system rewards. However, beyond a certain threshold (e.g., *Iterations* = 15), the system utility tends to stabilize. This is because, as training progresses, the agent accumulates more experience and knowledge, achieving a better balance between exploration and exploitation, resulting in a gradually stable system reward.

In Fig. 7(b)–(c), we present the performance of different methods in terms of latency and energy consumption within time slots. It can be observed that, except for the Random

algorithm, both latency and energy consumption of the system exhibit a decreasing trend over the iterations. In particular, Dueling DDQN consistently outperforms other algorithms for all scenarios. In our simulated setup, our proposed method demonstrates that both latency and energy consumption converge to a minimum value when QoE is maximized. Compared to the DDQN and PPO approaches, our dynamic Dueling DDQN solution achieves energy savings of 42.7% and 20.9%, respectively, and reduces energy consumption by 38.7% compared to the deep Q-network. Furthermore, our dynamic Dueling DDQN solution reduces latency by 10.5% and 19.6% when compared to the DDQN and PPO approaches respectively, and achieves a latency reduction of 29.9% compared to the deep Q-network.

The analyses of algorithm performance under maximum BEC bandwidth and secret sharing thresholds are provided in Appendices III and IV, respectively.

VI. CONCLUSION

The aim of this paper is to address the challenges of communication and computing resource demands, as well as data privacy in the Internet of Things. We propose a dynamic Blockchain-enabled Edge Computing IoT Network (i.e., BECIN) framework with dynamic entry and exit mechanism. In contrast to existing research, our framework considers the stochastic nature of task arrivals and the potential variability in the number, location, and resource supply of edge computing service providers. Our objective is to minimize system latency and energy consumption through a dynamic *Dueling DDQN* algorithm that is aware of the dynamically changing environment. We provide a comprehensive analysis of the convergence properties of the proposed dynamically self-adjusting BECIN framework model. Experimental results validate the feasibility and superior performance of the framework.

The primary challenges inherent in the proposed methodology lie in enhancing the system's adaptive capabilities, particularly in responding to complex and dynamic environments where real-time strategic adjustments are necessary to address fluctuating tasks and resource variations. Furthermore, as tasks and environments become increasingly diverse, current approaches may exhibit limitations in cross-domain and cross-task adaptability. Future research directions could explore the integration of transfer learning or meta-learning paradigms to augment the system's adaptive capabilities, enabling dynamic strategy optimization based on environmental and task-specific characteristics, thereby facilitating more efficient task allocation and resource management.

REFERENCES

- [1] Tuya, "Future development trends of the Internet of Things," May 15, 2023. [Online]. Available: <https://www.tuya.com/cn/industry-details/Kclhdc5y5kqiz>
- [2] Y. Zhang, X. Lan, J. Ren, and L. Cai, "Efficient computing resource sharing for mobile edge-cloud computing networks," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1227–1240, Jun. 2020.
- [3] F. S. Abkenar et al., "A survey on mobility of edge computing networks in IoT: State-of-the-art, architectures, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2329–2365, Fourth Quarter 2022.
- [4] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial Internet of Things: Architecture, advances and challenges," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2462–2488, Fourth Quarter 2020.
- [5] M. Sharma, A. Tomar, and A. Hazra, "Edge computing for industry 5.0: Fundamental, applications and research challenges," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 19070–19093, Jun. 2024.
- [6] H. Fang et al., "Empowering intelligent manufacturing with edge computing: A portable diagnosis and distance localization approach for bearing faults," *Adv. Eng. Inform.*, vol. 59, 2024, Art. no. 102246.
- [7] L. Xiao, D. Han, K.-C. Li, and M. K. Khan, "UETOPSIS: A data-driven intelligence approach to security decisions for edge computing in smart cities," *ACM Trans. Sensor Netw.*, 2024.
- [8] X. Chen, G. Zhu, H. Ding, L. Zhang, H. Zhang, and Y. Fang, "End-to-end service auction: A general double auction mechanism for edge computing services," *IEEE/ACM Trans. Netw.*, vol. 30, no. 6, pp. 2616–2629, 2022.
- [9] M. Mukherjee, R. Matam, C. X. Mavroumoustakis, H. Jiang, G. Mas-torakis, and M. Guo, "Intelligent edge computing: Security and privacy challenges," *IEEE Commun. Mag.*, vol. 58, no. 9, pp. 26–31, Sep. 2020.
- [10] Y. Wan, Y. Qu, L. Gao, and Y. Xiang, "Privacy-preserving blockchain-enabled federated learning for 5G-driven edge computing," *Comput. Netw.*, vol. 204, 2022, Art. no. 108671.
- [11] T. Li, H. Wang, D. He, and J. Yu, "Blockchain-based privacy-preserving and rewarding private data sharing for IoT," *IEEE Internet Things J.*, vol. 9, no. 16, pp. 15 138–15 149, Aug. 2022.
- [12] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving Byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2848–2861, 2022.
- [13] Y. Gao, Y. Chen, X. Hu, H. Lin, Y. Liu, and L. Nie, "Blockchain based IIoT data sharing framework for SDN-enabled pervasive edge computing," *IEEE Trans. Ind. Inform.*, vol. 17, no. 7, pp. 5041–5049, Jul. 2021.
- [14] L. Yuan et al., "CSEdge: Enabling collaborative edge storage for multi-access edge computing based on blockchain," *IEEE Transac. Parallel Distrib. Syst.*, vol. 33, no. 8, pp. 1873–1887, Aug. 2022.
- [15] Z. Xiong, J. Kang, D. Niyato, P. Wang, and H. V. Poor, "Cloud/edge computing service management in blockchain networks: Multi-leader multi-follower game-based ADMM for pricing," *IEEE Trans. Serv. Comput.*, vol. 13, no. 2, pp. 356–367, Mar./Apr. 2020.
- [16] Y. Huang, J. Zhang, J. Duan, B. Xiao, F. Ye, and Y. Yang, "Resource allocation and consensus of blockchains in pervasive edge computing environments," *IEEE Trans. Mobile Comput.*, vol. 21, no. 9, pp. 3298–3311, Sep. 2022.
- [17] H. Byeon, A. Bommiseti, M. Soni, and P. Khatri, "Enhancing security and privacy in wireless medical sensor networks through blockchain-enabled edge computing," in *Blockchain for Biomedical Research and Healthcare: Concept, Trends, and Future Implications*. Berlin, Germany: Springer, 2024, pp. 77–97.
- [18] Z. A. El Houda, H. Moudoud, B. Brik, and L. Khokhi, "Blockchain-enabled federated learning for enhanced collaborative intrusion detection in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7661–7672, Jul. 2024.
- [19] M. B. Mollah, M. A. K. Azad, and Y. Zhang, "Secure targeted message dissemination in iot using blockchain enabled edge computing," 2024, *arXiv:2401.06384*.
- [20] A. M. Rahmani, J. Tanveer, F. S. Gharehchopogh, S. Rajabi, and M. Hosseinzadeh, "A novel offloading strategy for multi-user optimization in blockchain-enabled mobile edge computing networks for improved internet of things performance," *Comput. Elect. Eng.*, vol. 119, 2024, Art. no. 109514.
- [21] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.
- [22] N. Zhang, S. Guo, Y. Dong, and D. Liu, "Joint task offloading and data caching in mobile edge computing networks," *Comput. Netw.*, vol. 182, 2020, Art. no. 107446.
- [23] T. Bai, C. Pan, Y. Deng, M. El-kashlan, A. Nallanathan, and L. Hanzo, "Latency minimization for intelligent reflecting surface aided mobile edge computing," *IEEE J. Areas Commun.*, vol. 38, no. 11, pp. 2666–2682, Nov. 2020.
- [24] C. You and K. Huang, "Multiuser resource allocation for mobile-edge computation offloading," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.

- [25] Y. Ding, C. Liu, K. Li, Z. Tang, and K. Li, "Task offloading and service migration strategies for user equipments with mobility consideration in mobile edge computing," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl. Big Data Cloud Comput. Sustain. Comput. Commun. Social Comput. Netw.*, 2019, pp. 176–183.
- [26] Z. Xu et al., "Energy-aware inference offloading for DNN-driven applications in mobile edge clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 4, pp. 799–814, Apr. 2021.
- [27] W. Fan, J. Han, L. Yao, F. Wu, and Y. Liu, "Latency-energy optimization for joint wifi and cellular offloading in mobile edge computing networks," *Comput. Netw.*, vol. 181, 2020, Art. no. 107570.
- [28] J. Luo, X. Deng, H. Zhang, and H. Qi, "QoE-driven computation offloading for edge computing," *J. Syst. Architecture*, vol. 97, pp. 34–39, 2019.
- [29] L. Li, T. Q. Quek, J. Ren, H. H. Yang, Z. Chen, and Y. Zhang, "An incentive-aware job offloading control framework for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 63–75, Jan. 2021.
- [30] X. Wang, Z. Ning, and S. Guo, "Multi-agent imitation learning for pervasive edge computing: A decentralized computation offloading algorithm," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 411–425, Feb. 2021.
- [31] Z. Ning et al., "Mobile edge computing enabled 5G health monitoring for Internet of Medical Things: A decentralized game theoretic approach," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 2, pp. 463–478, 2020.
- [32] H. Chen, Y. Ye, M. Xiao, M. Skoglund, and H. V. Poor, "Coded stochastic ADMM for decentralized consensus optimization with edge computing," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5360–5373, Apr. 2021.
- [33] "Online user allocation in mobile edge computing environments: a decentralized reactive approach," *J. Syst. Architecture*, vol. 113, 2021, Art. no. 101904.
- [34] L. Wang, Y. Xu, H. Xu, M. Chen, and L. Huang, "Accelerating decentralized federated learning in heterogeneous edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5001–5016, Sep. 2023.
- [35] L. Cui, S. Yang, Z. Chen, Y. Pan, Z. Ming, and M. Xu, "A decentralized and trusted edge computing platform for Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 3910–3922, May 2020.
- [36] Y. Liu, F. R. Yu, X. Li, H. Ji, and V. C. Leung, "Decentralized resource allocation for video transcoding and delivery in blockchain-based system with mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11 169–11 185, Nov. 2019.
- [37] L. Yuan et al., "Coopedge+: Enabling decentralized, secure and cooperative multi-access edge computing based on blockchain," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 894–908, Mar. 2022.
- [38] J. Guo, X. Ding, T. Wang, and W. Jia, "Combinatorial resources auction in decentralized edge-thing systems using blockchain and differential privacy," *Inf. Sci.*, vol. 607, pp. 211–229, 2022.
- [39] F. Sezgin, D. Vriesman, D. Steinhauser, R. Lugner, and T. Brandmeier, "Safe autonomous driving in adverse weather: Sensor evaluation and performance monitoring," in *Proc. IEEE Intell. Veh. Symp.*, 2023, pp. 1–6.
- [40] S. Vimal, M. Vadivel, V. V. Baskar, V. Sivakumar, and C. Srinivasan, "Integrating IoT and machine learning for real-time patient health monitoring with sensor networks," in *Proc. 4th Int. Conf. Smart Electron. Commun.*, 2023, pp. 574–578.
- [41] H. Jiang, X. Dai, Z. Xiao, and A. Iyengar, "Joint task offloading and resource allocation for energy-constrained mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4000–4015, Jul. 2023.
- [42] X. Deng, J. Li, L. Shi, Z. Wang, J. H. Wang, and T. Wang, "On dynamic resource allocation for blockchain assisted federated learning over wireless channels," in *Proc. IEEE Int. Conf. Internet Things IEEE Green Comput. Commun., IEEE Cyber Phys. Social Comput., IEEE Smart Data IEEE Congr. Cybermatics*, 2021, pp. 306–313.
- [43] D. C. Nguyen, S. Hosseinalipour, D. J. Love, P. N. Pathirana, and C. G. Brinton, "Latency optimization for blockchain-empowered federated learning in multi-server edge computing," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3373–3390, Dec. 2022.
- [44] R. Zhang, K. Xiong, Y. Lu, P. Fan, D. W. K. Ng, and K. B. Letaief, "Energy efficiency maximization in RIS-assisted SWIPT networks with rsma: A ppo-based approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 5, pp. 1413–1430, May 2023.
- [45] W. Guan, B. Gao, K. Xiong, and Y. Lu, "Spectrum sharing in UAV-assisted hetnet based on CMB-am multi-agent deep reinforcement learning," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2022, pp. 1–2.
- [46] D. Nguyen, M. Ding, P. Pathirana, A. Seneviratne, J. Li, and V. Poor, "Cooperative task offloading and block mining in blockchain-based edge computing with multi-agent deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2021–2037, Apr. 2023.
- [47] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, Jun. 2020.



Qiang He received the PhD degree in computer application technology from Northeastern University, Shenyang, China, in 2020. He also worked with the School of Computer Science and Technology, Nanyang Technological University, Singapore, as a visiting Ph.D. Researcher from 2018 to 2019. He is currently a professor with the College of Computing and Artificial Intelligence, Northeastern University, Shenyang, China. His research interests include cloud computing, edge computing, machine learning, social network analytics, and health care. He has published more than 90 journal articles and conference papers, including those in *IEEE/ACM Transactions on Networking*, *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Neural Networks and Learning Systems*, *NeurIPS*, and *AAAI*.



Zheng Feng received the master's degree from Northeastern University in 2024. His main research interests include mobile Edge computing, blockchain, and artificial intelligence.



Hui Fang received the PhD degree from Nanyang Technological University, Singapore. She is currently a professor with the Shanghai University of Finance and Economics, China, affiliated with the Research Institute for Interdisciplinary Sciences and the Key Laboratory of Interdisciplinary Research of Computation and Economics. Her research interests include personalized machine learning, with a focus on trust and link prediction in online communities, as well as recommender systems. She has published extensively in leading conferences such as *IJCAI*, *AAAI*, and *SIGIR*, and in top journals including *AIJ*, *IEEE Transactions on Knowledge and Data Engineering*, *TOIS*, and *IEEE Transactions on Pattern Analysis and Machine Intelligence*. She serves as a senior editor for the *ECRA* journal, and as a Program Committee Board Member for *IJCAI*, in addition to serving as a (Senior) Program Committee Member for *WWW*, *UMAP*, *AAAI*, *AAMAS*, and other major venues.



Xingwei Wang (Member, IEEE) received the BS, MS, and PhD degrees in computer science from the Northeastern University, Shenyang, China, in 1989, 1992, and 1998, respectively. He is currently a professor with the College of Computer Science and Engineering, Northeastern University, Shenyang, China.



Liang Zhao received the PhD degree from the School of Computing, Edinburgh Napier University, in 2011. He is a professor with Shenyang Aerospace University, China. He is a member of IEEE INGR SatCom and IEEE ComSoc SSC. Before joining Shenyang Aerospace University, he worked as associate senior Researcher in Hitachi (China) Research and Development Corporation from 2012 to 2014. His research interests include ITS, VANET, WMN and SDN. He has published more than 120 papers. He served as the

Chair of several international conferences and workshops, including 2021 IEEE TrustCom (Program Co-Chair), 2019 IEEE IUCC (Program Co-Chair), and 2018-2021 NGDN workshop (founder). He is associate editor of *Frontiers in Communications and Networking* and *Journal of Circuits Systems and Computers*. He is/has been a guest editor of *IEEE Transactions on Network Science and Engineering*, *Springer Journal of Computing*, etc. He was the recipient of the Best/Outstanding Paper Awards at 2015 IEEE IUCC, 2020 IEEE ISPA, and 2013 ACM MoMM.



Kim-Kwang Raymond Choo (Senior Member, IEEE) holds the Cloud Technology Endowed professorship with the University of Texas at San Antonio. His awards include ESORICS 2015 Best Research Paper Award, 2015 Winning Team of Germany's University of Erlangen-Nuremberg Digital Forensics Research Challenge, 2014 Australia New Zealand Policing Advisory Agency's Highly Commended Award, 2010 Australian Capital Territory Pearcey Award, Fulbright Scholarship, 2008 Australia Day Achievement Medallion, and British Computer Society's Wilkes Award. He is an Australian Computer Society Fellow.



Keping Yu (Member, IEEE) received the ME and PhD degrees from the Department of Computer Science and Communications Engineering, Faculty of Science Engineering, Waseda University, Tokyo, Japan, in 2012 and 2016, respectively. From 2015 to 2019, he was a research associate with the Global Information and Telecommunication Institute, Waseda University, Tokyo, Japan. Since 2019, he has been a junior researcher (assistant professor) with the Global Information and Telecommunication Institute, Waseda University. He has hosted and participated in

many research projects with Ministry of Internal Affairs and Communication (MIC), Japan, Ministry of Economy, Trade and Industry, Japan, Japan Society for the Promotion of Science, Advanced Telecommunications Research Institute International (ATR), Japan, Keihin Electric Railway Corporation, Japan, and Maspro Denkoh Corporation, Japan. His research interests include smart grids, information-centric networking, the Internet of things, and information security. He also served as a TPC Member for IEEE VTC. He is a member of IEICE.